

DOCUMENT 3

INNOMINE TECHNICAL ARCHITECTURE DOCUMENT (TAD)

Version: 1.0

Status: Draft

Project Name: Innomine

Prepared By: Mohammed Kazim Khan & OpenAI ChatGPT

Purpose of this Document

This Technical Architecture Document (TAD) defines the engineering blueprint for Innomine. It specifies the system architecture, technology stack, project structure, coding standards, backend design, database architecture, security model, deployment strategy, scalability principles, and software engineering practices that shall govern the implementation of the platform.

This document serves as the primary technical reference for software engineers, architects, DevOps engineers, AI coding agents, and future contributors. All development activities shall align with the standards, principles, and architectural decisions defined herein.

1. Architecture Vision

Innomine shall be engineered as a modern, scalable, secure, and maintainable web platform capable of supporting long-term growth from an MVP to an enterprise-grade innovation marketplace.

The architecture shall prioritize:

- Scalability
- Maintainability
- Performance
- Security
- Modularity
- Reusability
- Developer Experience
- Cloud Readiness
- AI Integration Readiness

The system shall be designed to accommodate millions of users, products, and media assets without requiring major architectural redesign.

2. Engineering Principles

The following principles shall guide all technical decisions:

Separation of Concerns

Business logic, presentation, data access, and infrastructure shall remain independent.

Modular Architecture

Features shall be organized into independent modules to simplify maintenance and future expansion.

API-First Design

Frontend and backend shall communicate exclusively through well-defined APIs, enabling independent development and future mobile application support.

Reusability

Common functionality shall be abstracted into reusable services, utilities, and components.

Scalability

The system shall be designed to scale horizontally and support future service decomposition if required.

Security by Design

Security shall be considered during every phase of development rather than being added after implementation.

Clean Code

All code shall prioritize readability, consistency, documentation, and maintainability.

3. Technology Stack

The technology stack has been selected based on performance, ecosystem maturity, community support, scalability, and developer productivity.

Frontend

- Next.js (Latest Stable Version)
- React 19

- TypeScript
 - Tailwind CSS
 - Shadcn/UI
 - Radix UI
 - Framer Motion
 - Zustand
 - TanStack Query
 - React Hook Form
 - Zod
 - Axios
 - Lucide React
-

Backend

- Node.js (LTS)
 - Express.js
 - TypeScript
 - JWT Authentication
 - Bcrypt
 - Multer
 - Socket.IO
-

Database

- MongoDB Atlas
 - Mongoose ODM
-

Media Storage

- Cloudinary
-

Payments

- Razorpay
- Stripe

Deployment

Frontend

- Vercel

Backend

- Render

Database

- MongoDB Atlas

Media

- Cloudinary
-

Development Tools

- Git
 - GitHub
 - VS Code
 - Postman
 - Figma
 - ESLint
 - Prettier
 - Husky
 - GitHub Actions (Future)
-

4. High-Level System Architecture

The platform shall follow a layered three-tier architecture.

Presentation Layer

|

|— Next.js Frontend

|— Responsive UI

|— Authentication

|— Dashboards

|— Innovation Feed

|



Application Layer

|

├— REST APIs

├— Authentication Service

├— Product Service

├— Reel Service

├— Order Service

├— Notification Service

├— Payment Service

|



Data Layer

|

├— MongoDB

├— Cloudinary

├— Future Redis Cache

5. Architectural Characteristics

The architecture shall provide:

- Loose Coupling
- High Cohesion
- Stateless APIs
- Cloud-Native Deployment
- Modular Feature Organization
- Centralized Configuration
- Dependency Injection (where beneficial)
- Event-Driven Expansion Capability (Future)

6. Application Architecture

The application shall be divided into four major systems:

Public Website

Responsible for:

- Landing Page
 - Marketplace
 - Categories
 - Search
 - Innovation Feed
 - Product Pages
 - Blogs
 - Contact
-

Buyer Portal

Responsible for:

- Orders
 - Wishlist
 - Cart
 - Checkout
 - Profile
 - Notifications
-

Innovator Portal

Responsible for:

- Storefront
 - Product Management
 - Reel Management
 - Analytics
 - Orders
 - Verification
-

Administration Portal

Responsible for:

- User Management
 - Moderation
 - Reports
 - CMS
 - Verification
 - Analytics
 - Platform Configuration
-

7. System Design Principles

The architecture shall support:

- Independent frontend deployment
 - Independent backend deployment
 - API versioning
 - Horizontal scalability
 - Future mobile applications
 - Third-party integrations
 - AI-powered services
 - Modular feature development
-

INNOMINE TECHNICAL ARCHITECTURE DOCUMENT (TAD)

8. Project Directory Structure

The project shall be organized as a **Monorepo** containing independent frontend and backend applications. This separation enables independent deployment, testing, and scaling while maintaining a unified development workflow.

innomine/

└─ frontend/

|

└─ backend/

|

└─ docs/

```
|
|
|— shared/
|
|— scripts/
|
|— .github/
|
|— docker/
|
|— README.md
```

8.1 Frontend Structure

The frontend shall use **Feature-Sliced Design** with clear separation between application-level configuration, reusable components, and business features.

```
frontend/
src/
app/
components/
features/
layouts/
pages/
hooks/
services/
store/
contexts/
types/
utils/
assets/
styles/
constants/
config/
```


middleware/

Frontend Design Principles

- Feature-based organization.
 - Reusable UI components.
 - Clear separation between business logic and presentation.
 - Shared component library.
 - API abstraction layer.
 - Type-safe development.
-

8.2 Backend Structure

The backend shall follow a **Feature-Based Modular Architecture**.

backend/

src/

modules/

auth/

users/

products/

categories/

reels/

innovationVerification/

orders/

cart/

wishlist/

payments/

reviews/

comments/

notifications/

analytics/

admin/

dashboard/

shared/

config/
database/
middlewares/
utils/
types/
constants/
jobs/
server.ts

Each module shall contain:

products/
product.controller.ts
product.service.ts
product.routes.ts
product.model.ts
product.validation.ts
product.repository.ts
product.types.ts

This ensures that every feature remains self-contained and independently maintainable.

9. Frontend Architecture

The frontend shall adopt a **Component-Driven Architecture**.

Component Categories

UI Components

Reusable design system elements.

Examples:

- Button
- Card
- Modal
- Input
- Badge

- Avatar
 - Tooltip
-

Business Components

Domain-specific components.

Examples:

- ProductCard
 - ReelPlayer
 - SellerCard
 - ProductGallery
 - InnovationTimeline
 - CheckoutSummary
-

Layout Components

Responsible for page structure.

Examples:

- Navbar
 - Footer
 - Sidebar
 - DashboardLayout
 - AuthLayout
-

Page Components

Top-level pages rendered by Next.js.

Examples:

- Home
- Explore
- Product Details
- Feed
- Seller Dashboard
- Admin Dashboard

10. Backend Architecture

The backend shall follow a layered architecture.

Each request shall flow through the following layers:

Client

↓

Routes

↓

Controller

↓

Service

↓

Repository

↓

Database

Route Layer

Responsible for:

- API endpoints
- Middleware
- Authentication
- Validation

No business logic.

Controller Layer

Responsible for:

- Receiving requests
- Calling services
- Returning responses

Controllers remain lightweight.

Service Layer

Contains:

Business Logic

Examples:

- Price calculations
 - Order validation
 - Recommendation logic
 - Inventory updates
 - Payment processing
-

Repository Layer

Responsible only for database operations.

Examples:

- Find User
- Save Product
- Delete Reel
- Update Order

No business logic.

Database Layer

MongoDB collections.

No application logic.

11. State Management Architecture

The frontend shall separate state into distinct categories.

Local UI State

Managed using:

React State

Examples:

- Modal open

- Dropdown
- Search input

Global Client State

Managed using:

Zustand

Examples:

- Logged-in user
- Theme
- Shopping cart
- Notifications

Server State

Managed using:

TanStack Query

Examples:

- Products
- Orders
- Feed
- Categories

Form State

Managed using:

React Hook Form

Validation:

Zod

12. API Communication Architecture

Communication shall follow REST principles.

Frontend

↓

Axios Client

↓

API Layer

↓

Backend

↓

Database

The frontend shall never access the database directly.

API Response Standard

Every API shall return a consistent structure.

Success Response

```
{  
  "success": true,  
  "message": "Operation completed successfully",  
  "data": {}  
}
```

Error Response

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": []  
}
```

13. Authentication Architecture

Authentication shall be implemented using JWT.

Workflow:

Register

↓

Email Verification

↓

Login

↓

JWT Access Token

↓

Refresh Token

↓

Protected APIs

Features:

- Secure password hashing
 - Email verification
 - Refresh tokens
 - Role-based authorization
 - Session management
 - Password reset
 - Secure logout
-

14. Database Architecture

- Collections
- Relationships
- Indexing
- Validation
- Soft Deletes
- Backups

15. Infrastructure, Deployment & Scalability

- Vercel
- Render
- MongoDB Atlas
- Cloudinary
- CDN
- Scaling
- Performance

- Caching

16. Development Standards & Best Practices

- Coding Standards
- Git Workflow
- Branch Naming
- Commit Convention
- Documentation
- Error Handling
- Logging
- Testing
- Environment Variables

17. Architecture Summary & Future Evolution

- Final architecture recap
- Future migration plans
- AI integration readiness
- Mobile application readiness
- Microservices roadmap (future, not now)

This is the final structure.